

Aula 8 — Implementação de um Analisador Léxico

Aula prática (laboratório)

Compiladores I

FACOM - Faculdade de Computação

Bloco 3: Análise Léxica

Duração estimada: 70–80 minutos

Revisão do bloco de Análise Léxica

- ▶ Diagramas de transição → autômatos finitos (AFD/AFN) → estrutura de um scanner real;
- ▶ `tokens.h`, atributos e tratamento de erros léxicos do Micro C.

Pergunta rápida

Em uma frase: o que um scanner recebe como entrada e o que ele entrega como saída?

Objetivos de aprendizagem

- ▶ Implementar um analisador léxico funcional para reconhecer os tokens de uma linguagem simples;
- ▶ Tratar erros léxicos básicos;
- ▶ Testar e depurar o analisador léxico implementado.

Sumário da aula

O scanner do Micro C

A função proximo_token()

Exemplo — Esqueleto simplificado (Cap. 2.5.2)

```
1 Token proximo_token(void) {
2     pular_espacos_e_comentarios();
3     if (eh_letra(char_atual))
4         return ler_identificador_ou_palavra_chave();
5     if (eh_digito(char_atual))
6         return ler_numero();
7     if (char_atual == '=')
8         return ler_igual_ou_atribuicao();
9     /* ... outros casos ... */
10    return token_erro(char_atual);
11 }
```

Desafios práticos comuns (Cap. 2.5.3)

- ▶ **Lookahead:** decidir entre `=` e `==` exige olhar o próximo caractere antes de decidir o token;
- ▶ **Buffer/posição:** controlar a posição atual no código-fonte (linha, coluna) para mensagens de erro úteis;
- ▶ **Palavras reservadas:** reconhecer como identificador primeiro, depois comparar com uma tabela de palavras-chave;
- ▶ **Fim de arquivo:** tratar corretamente o token especial `TK_EOF`.

Roteiro da atividade prática

1. Formem duplas ou trios;
2. Implementem (em C, pseudocódigo ou fluxograma) uma função que reconheça pelo menos: identificadores, números inteiros, os operadores `+` `-` `*` `/` `=` e o delimitador `;`;
3. Testem com a entrada: `soma = a + b * 2;`
4. Anotem a sequência de tokens gerada.

Desafio em sala

Adicionem suporte a comentários de linha (`//...`) que devem ser ignorados pelo scanner.

Testando e depurando

- ▶ Sempre teste casos de borda: string vazia, apenas espaços, número seguido de letra (123abc);
- ▶ Gere uma saída legível: liste (token, lexema) para cada token reconhecido — é exatamente o artefato que o Micro C produz ao final da análise léxica;
- ▶ Se um caractere não corresponde a nenhum padrão, gere um erro léxico claro, indicando a posição.

Exemplo — Saída esperada para `soma = a + b * 2;`

```
(TK_ID,soma) (TK_ASSIGN,=) (TK_ID,a) (TK_PLUS,+)
(TK_ID,b) (TK_TIMES,*) (TK_NUM,2) (TK_SEMI,;)
```

Síntese da aula

- ▶ A implementação real de um scanner combina os autômatos estudados com detalhes práticos (lookahead, posição, palavras reservadas);
- ▶ Testar com casos de borda é essencial para um scanner robusto;
- ▶ Encerramos aqui o bloco de Análise Léxica.

Próxima aula: início do bloco de Análise Sintática — visão geral entre análise descendente e ascendente.

Teste rápido — Aula 8

Questões teóricas

1. Por que o tratamento de palavras reservadas é feito *depois* de reconhecer o padrão de identificador?
2. O que é o token `TK_EOF` e por que ele é necessário?

Questão prática

3. Dada a entrada `x = 10 // comentário\ny = 20;`, liste a sequência de tokens que um scanner correto deveria gerar.